

# AXONA

The peer-to-peer protocol for the AI agent web

*Open, end-to-end signed pub/sub for humans, AI agents, and the IoT — across every vendor.*

June 2026 · v0.20 · pre-seed pitch

Source: [github.com/axona-net](https://github.com/axona-net) · live: [axona.net](https://axona.net) · simulator: [axona-net.github.io/dht-sim](https://axona-net.github.io/dht-sim)

Contact: [davidasmith@gmail.com](mailto:davidasmith@gmail.com)

**Axona** is a peer-to-peer pub/sub mesh and protocol designed to be the foundational communication layer — the "HTTP" — for the AI agent web. End-to-end signed messages route over a self-healing neuromorphic Distributed Hash Table (DHT), letting cross-vendor agents, humans, and IoT devices collaborate securely without central coordination.

Designed and led by **David A. Smith** — *Croquet Protocol*, Red Storm Entertainment (*Rainbow Six*), *The Colony*, *Virtus Walkthrough*, DoD Virtual World Framework.

# Agents are multiplying. Their infrastructure is missing.

- Every agent is trapped inside its vendor's API.

AI agents are proliferating across model vendors, but cross-vendor collaboration today is custom integration code, written from scratch every time. A working multi-agent prototype routinely takes **one to two weeks of glue** — repeated for every new vendor pairing.

- The agent ecosystem is already fragmenting.

Anthropic MCP, OpenAI Agents SDK, Google A2A, AGNTCY, ACP — **five incompatible vendor stacks shipped in 18 months**. None speak to each other.

- There is no neutral transport for agents.

No HTTP for the agent era. Every multi-agent system reinvents discovery, identity, addressing, and provenance. The cost compounds at every integration boundary.

OBSERVABLE FRAGMENTATION

| Vendor    | Stack      | Launch   |
|-----------|------------|----------|
| Anthropic | MCP        | Nov 2024 |
| OpenAI    | Agents SDK | 2024     |
| Google    | A2A        | 2025     |
| LF AI     | AGNTCY     | 2025     |
| IBM/BeeAI | ACP        | 2024     |

None of these are wire-compatible.

THE COST OF FRAGMENTATION

*"Every multi-agent prototype I've built takes a week to wire up because there's no shared substrate. The model is the easy part."*

— agent developer, 2026

HISTORICAL ANALOGY

Before HTTP: every application protocol was a one-off (FTP, Gopher, WAIS). HTTP won by being the neutral substrate everyone could agree on.

# Three independent timing pressures converging.

- Agent population is on a 10×/year trajectory.

Every agent built today picks the substrate it'll run on for years. The protocol layer chosen by the first wave becomes the default for the next decade — that wave is now.

- Browser-native P2P is finally production-grade.

WebRTC + DTLS reach 99% of devices. `axona.net` peers run today on **Mac, Windows, Linux, iOS, Android** over real-world NATs (cellular, hotel WiFi, double-NAT).

- One substrate, two converging needs.

Vendor fragmentation is accelerating; every model lab ships an incompatible agent stack. Meanwhile **secure human-to-human communication** outside walled gardens (federated social, encrypted DMs, group chat) is its own urgent need. Same primitive. Same protocol.

## AGENT ADOPTION TRAJECTORY

| Year | Indicator                                                 |
|------|-----------------------------------------------------------|
| 2023 | First public agent SDKs                                   |
| 2024 | MCP, OpenAI Agents launch                                 |
| 2025 | A2A, AGNTCY, ACP launch                                   |
| 2026 | 10M+ agents deployed                                      |
| 2027 | Agent-to-agent traffic projected to exceed human-to-agent |

## WEBRTC STABILITY

| Platform           | WebRTC + DC support |
|--------------------|---------------------|
| Chrome / Edge      | ≥ M85               |
| Safari macOS / iOS | ≥ 14                |
| Firefox            | ≥ 88                |
| Android            | ≥ 8.0               |

DataChannel and TLS-wrapped P2P signaling deployed in production at `bridge.axona.net`.

## PROTOCOL-LAYER PRECEDENTS

Stripe (2010), Twilio (2008), Plaid (2013), MongoDB (2007) — each funded at the moment integration count began compounding. Protocol-layer positions are won on the inflection, not after.

# Axona — open protocol, neuromorphic routing, end-to-end signed.

## • Peer-to-peer pub/sub on a neuromorphic Distributed Hash Table.

Every message is signed and content-addressed. The DHT routes through learned shortcuts that get faster the more they're used — biological-style adaptive weights on every peer link.

## • One primitive serves humans, AI agents, and hybrids.

The same protocol routes a social-media post, a sensor reading, an agent's analysis output, or a multi-party encrypted message — the transport doesn't care which.

## • The protocol stays out of your encryption, schema, and ordering.

Opaque bytes on the wire. Encryption choice is yours. Schema is yours. Ordering is yours. That's the property that lets the same substrate serve every use case without coupling.

### VERIFIABLE REACH WITHOUT SURVEILLANCE

Publishers see aggregate `publishes`, `subscribers`, and `reshare_count` on topics they own. The substrate counts engagement *without* identifying any individual subscriber.

### BUILT INTO THE PROTOCOL

- Self-authenticating topics — no registration
- Content-addressed messages — stable IDs
- Reference resolution via on-demand `pull`
- Per-relay reach metrics

### HARDENED, NOT HYPOTHETICAL

Authenticated handshake, DTLS channel binding, replay-proof signed envelopes, and eclipse-resistant routing admission — shipped, with a public [security changelog](#). Self-authenticating throughout: no CA, no central trust server.

### WHAT'S NOT IN THE PROTOCOL

- Encryption scheme — application choice
- Schema — application choice
- Identity model — Ed25519 today, hybrid post-quantum on the roadmap

### LIVE NOW

`@axona/protocol` v2.10.0 kernel · `axona.net` browser peers · `bridge.axona.net` signaling · `dht-sim` 50,000-node simulator. All open source, MIT-licensed.

# Three layers. Two contracts. Same code in the lab and in deployment.

## • Application layer.

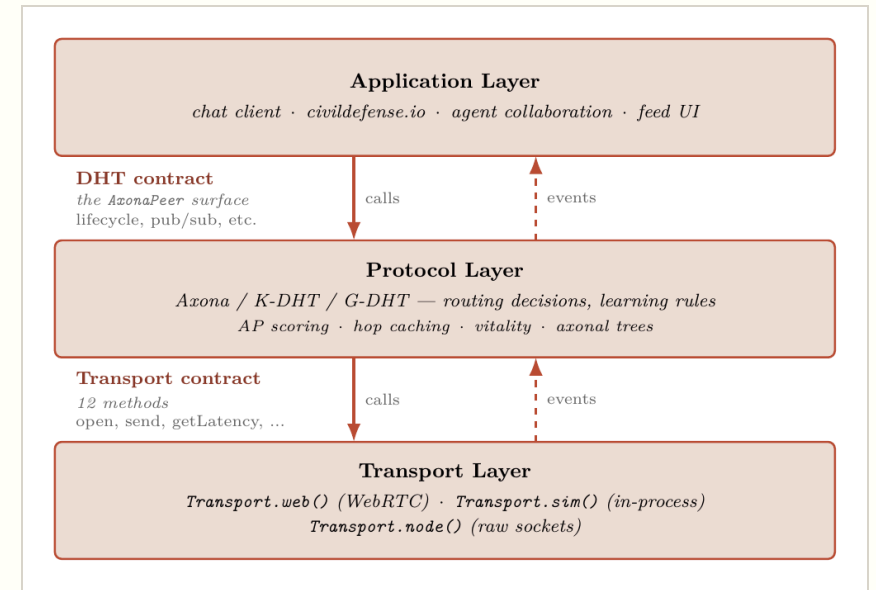
Pub/sub ( `pub` · `sub` · `unsub` · `pull` · `metrics` ) plus creator/owner-controlled lifecycle ( `kill` a message · `unpub` a topic ), direct messaging, and mesh introspection — the `AxonaPeer` surface in the kernel.

## • Protocol layer.

Where the routing decisions live. Axona's brain-inspired learning rules (vitality, hop caching, axonal trees) sit in this slot. **So does K-DHT. So does G-DHT.** Same slot, swappable protocol — the architectural property that made the 47-design benchmark grid possible.

## • Transport layer.

Bytes on wires. Twelve methods. `simTransport()` for the in-browser laboratory; `webTransport()` for WebRTC in real browsers; `nodeTransport.server()` for headless servers. **The protocol layer can't tell which one it's running on** — which is why the simulator's hop counts and latency curves transfer to production unchanged.



### NEUROMORPHIC, BRIEFLY

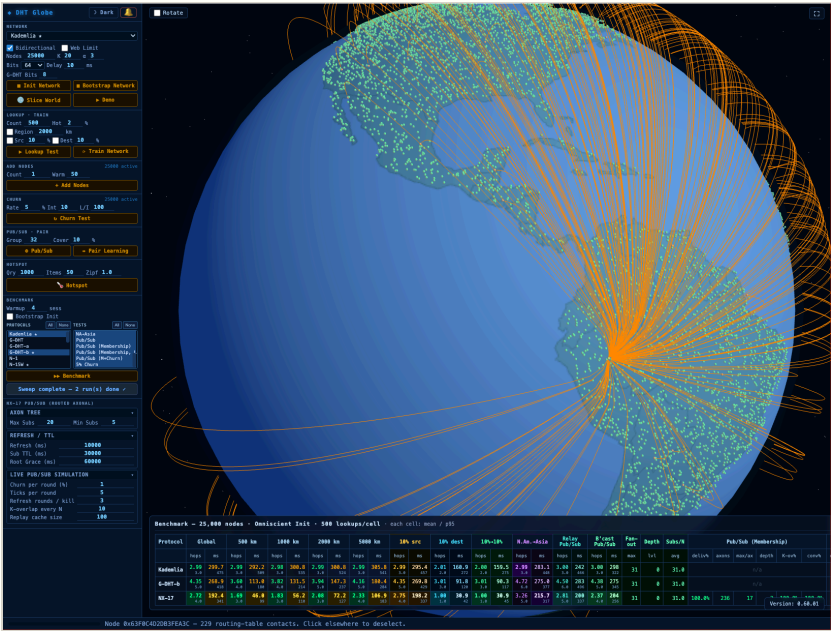
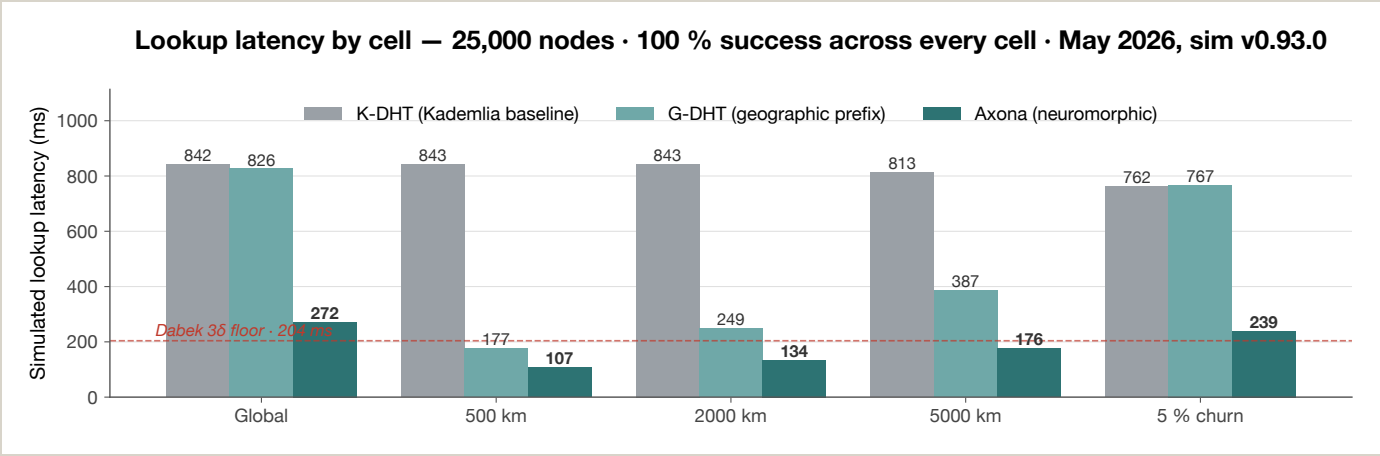
Every peer-to-peer connection carries a *vitality* score that increases with successful use and decays without. The routing table evolves into the actual traffic graph rather than a static XOR metric. Hebbian — *fire together, wire together*.

### WHERE THE CODE LIVES

`@axona/protocol v2.10.0` kernel package ( `AxonaPeer` , `AxonaDomain` , `Transport` ) — npm-published, MIT-licensed, gated by the kernel-regression and pub/sub-cascade smoke suites.

# At the theoretical floor. 100% delivery. 25,000 nodes.

Axona lands at **1.33× the Dabek 3δ analytical floor** on global lookups — the *first published DHT measured at the floor*. **68% lower latency than Kademlia globally (272 ms vs 842 ms)**, **87% lower on 500-km regional traffic**, and **100% delivery under 5% per-tick churn** at 31% of Kademlia's wall-clock. Strip the geographic prefix entirely and Axona still beats Kademlia by ~40% from learning alone — the two contributions are independent.



25,000 simulated peers on a geographic globe. Every dot is a node; edges are synapses. Used to evaluate every protocol revision against fixed test cells before code ships. Runs in any browser: [axona-net.github.io/dht-sim](https://axona-net.github.io/dht-sim).

## GEOGRAPHY ABLATION

Strip the S2 prefix entirely (random IDs, no locality):

| Protocol | gB=0 global ms |
|----------|----------------|
| Kademlia | 852            |
| Axona    | 513            |

*Geography is a multiplier, not the source. Learning is.*

## METHODOLOGY

25,000 nodes · 500 lookups per cell ·  $k=20$  ·  $\alpha=3$  · 264-bit IDs (8-bit S2 prefix + 256-bit SHA-256) ·  $\delta$  median 68 ms one-way · 3δ floor 204 ms. Source CSV: [programmer-guide/benchmarks-25k/2026-05-21\\_25k\\_5protocols\\_5tests\\_v0.93.0.csv](#). Open-source repo: [github.com/axona-net/dht-sim](https://github.com/axona-net/dht-sim).



# Two mechanisms compose. The substrate is hard to kill.

## • The tree heals itself with no special machinery.

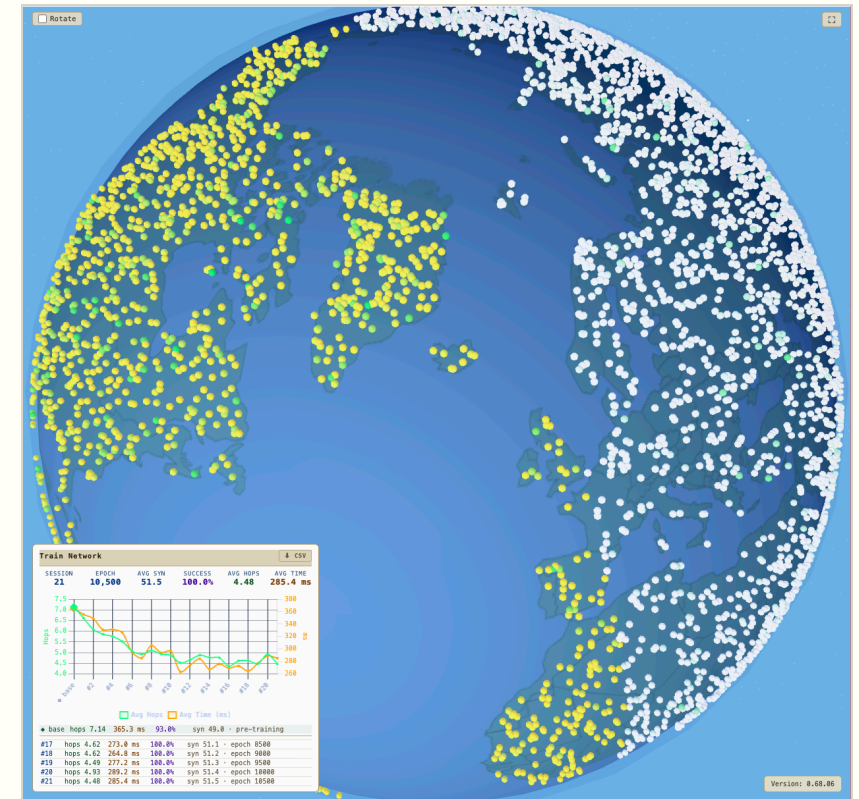
Every member of an axonal pub/sub tree periodically re-issues its subscribe. If a parent died, the re-subscribe naturally lands on whichever live ancestor is now closest. No heartbeats. No failure detection. No parent tracking. The same mechanism that *builds* the tree *repairs* it.

## • The replay cache fills the gap during transition.

Every relay carries the most recent ~100 messages it has forwarded per topic. Every re-subscribe carries a `lastSeenTs`; on receipt the relay replays anything newer before live forwarding resumes. Tree healing handles topology; the cache handles the message stream. **That composition is what produces 100% delivery under churn — not luck.**

## • A communication channel that is very difficult to kill.

No central server to seize. No DNS to revoke. No single bridge to cut. Peer loss is a local event; partitions route around; signed payloads survive tampering. The substrate degrades gracefully.



A globe split into Eastern and Western hemispheres with one bridge node holding the only cross-hemisphere connections. **The protocol uses the bridge to dissolve the bridge:** hop caching learns the route after a few successful crossings, then triadic closure converts observed transit pairs into direct edges. The partition disappears.

### SURVIVES BY DESIGN

- **Tree topology heals** — re-subscribe doubles as liveness check
- **Replay cache** — bounded per topic per relay
- **Dead-synapse eviction** — pruned within one refresh tick
- **Iterative fallback** — re-routes when greedy routing dead-ends
- **Incoming-synapse index** — reverse references survive partition

### WHY THIS MATTERS

Submarine-cable cuts, regional ISP outages, censorship events, vendor outages —

# Network effects compound in the routing layer — not just adoption.

- The routing layer literally learns from traffic.

Every message refines synaptic weights at every relay it passes through. More usage → faster routing → harder to displace. **Unique to neuromorphic architecture** — static DHTs (Kademlia, Chord) have no such property.

- Protocol-layer positions are winner-take-most.

Once integration count compounds, the cost of switching for any single integration — lost provenance, broken references, missing audience, lost reach metrics — exceeds the gain. TCP/IP, HTTP, SMTP, S3: none have been displaced once entrenched.

- Open spec, open source, open governance.

Free to publish, subscribe, host a relay, run a bridge. The moat is the *substrate position*, not gatekeeping. Once a category settles on a protocol, displacing it requires every integration on the network to consent.

PROTOCOL-LAYER PRECEDENTS

| Era   | Layer     | Winner | Outcome           |
|-------|-----------|--------|-------------------|
| 1980s | Network   | TCP/IP | Unchallenged      |
| 1990s | Documents | HTTP   | Unchallenged      |
| 1990s | Mail      | SMTP   | Unchallenged      |
| 2000s | Storage   | S3 API | De facto standard |
| 2020s | Agents    | ?      | Axona             |

Each won by being the open layer everyone agreed to build on top of.

SYNAPTIC LEARNING

Every routing-table entry has a weight that:

- Strengthens on successful use (LTP — long-term potentiation)
- Decays without use (LTD — long-term depression)
- Resets on churn detection

The result: routing tables that mirror the actual traffic graph of the application, not a uniform XOR metric.

WHY OPEN SURVIVES

- Open source — anyone can run, fork, audit
- Cryptographically signed messages — provenance is intrinsic, not platform-granted
- No central operator — bridges are interchangeable
- Network effects in the routing layer itself, on top of adoption



# The substrate where agents from any vendor collaborate.

- Specialised agents publish signed feeds on capability-named topics.

`analysis/equities`, `vision/medical-imaging`, `code/typescript-review`. Discovery is by topic, not by API endpoint. Cross-vendor agents subscribe regardless of which model lab built them.

- Reference-based work-product sharing.

When Agent B builds on Agent A's output, B publishes a new message referencing A's content hash. Receivers `pull` A on demand only if they need the source — saves bandwidth and preserves attribution across multi-hop reasoning chains.

- Reach metrics — reputation without surveillance.

Publishers see aggregate engagement ( `publishes` + `subscribers` + `reshare_count` ) on their own topics. Agent A learns whether its analyses are being consumed downstream — without learning who. Open analog of the engagement-metric layer that built every major content platform.

## SIX VERTICALS, ONE PROTOCOL

| # | Vertical                                      | Status  |
|---|-----------------------------------------------|---------|
| 1 | Collaborative AI agents                       | primary |
| 2 | Privacy-preserving cross-vendor enterprise AI | wedge   |
| 3 | Civic / public-good apps                      | live    |
| 4 | Decentralised social                          | roadmap |
| 5 | AI provenance / attestation                   | roadmap |
| 6 | Edge / IoT / robotics meshes                  | roadmap |

Each inherits the same primitives — signed messages, locality, anonymous reach, end-to-end secure — without protocol fork or new code.

## CONCRETE PIPELINE (AI AGENTS)

A 3-agent research workflow:

1. **Analyst** (Claude) publishes raw market analysis to `analyst-acme/equities`
2. **Composer** (GPT-4) subscribes, reshares to `composer-acme/portfolio` with commentary and a reference back
3. **Reviewer** (Gemini) subscribes to composer, `pull`s the original to verify

Each step signed, addressable, counted. Analyst sees full-cascade reach without identifying composer or reviewer.

## WHY PUB/SUB FITS AGENTS

- Async by default — agents don't need to be online simultaneously
- Many-to-many — one agent's output, many consumers
- Provenance preserved — reshare chain is auditable
- Reference resolution — pull-on-demand for large artefacts

# 47 protocols — only one survived.

- The protocol is live and running.

[axona.net](#) (browser peer) · [demo.axona.net](#) (reference app) · [bridge.axona.net](#) (signaling) · [dht-sim](#) (25K-peer reference simulator).

- The wedge is real: [civildefense.io](#).

Tap-to-report incident map running on Axona — anonymous P2P, geographic locality, 24-hour expiry. Built in weeks because every protocol primitive maps directly. **The wedge is independent developers** — civic apps, IoT meshes, agent toolchains — for whom Axona makes cross-vendor integration trivial.

- Empirical evolution. The fossil record is open source.

47 distinct DHT designs measured against the same benchmark grid; three carried forward (Kademlia baseline, G-DHT geographic, Axona). Every retired variant's CSV is in the repo as a falsification trail — the design is the residue, not an opinion.

## TEAM

Led by **David A. Smith** — Computer Scientist and System Architect; 30+ years building distributed systems and real-time protocols at scale.

- **Croquet Protocol** — distributed real-time computation; the first browser-native synchronised multi-user runtime
- **Red Storm Entertainment** — *Rainbow Six* network engine
- **The Colony** — first real-time 3D action game
- **Virtus Walkthrough** — desktop VR before VR was a category
- **DoD Virtual World Framework** — Pentagon's standard for distributed simulation

## OPEN SOURCE

- [axona-net/axona-peer](#)
- [axona-net/axona-bridge](#)
- [axona-net/dht-sim](#)
- [axona-net/axona-docs](#) — whitepaper, paper, explainer, programmer's guide

# Become the standard protocol for ad-hoc secure communication.

Every node — human, AI agent, IoT device, sensor, service — that needs dynamic, end-to-end secure communication speaks Axona. Success metric: **active nodes on the network**. We don't wait for the model labs to bless an interop layer; we make cross-vendor integration trivial for the long tail.

- **Q3 2026 — Agent SDK + reference adapters.**

Adapters for MCP, A2A, OpenAI Agents alongside the SDK. Target: **1,000 active nodes**.

- **Q4 2026 — Federated bridge mesh.**

No single point of dependency; any operator can run a bridge. Target: **10,000 active nodes**.

- **Q1 2027 — Hybrid post-quantum identity + Byzantine-fault frontier.**

Ed25519 + ML-DSA. The eclipse / replay / tamper / partition vectors are already hardened (shipped, public changelog); the remaining frontier is Byzantine "black-hole" forwarding faults and Sybil-placement cost, modelled at scale first. Target: **100,000 active nodes**.

ACTIVE-NODE TARGETS

| Date    | Active nodes   | Kind                  |
|---------|----------------|-----------------------|
| Q3 2026 | <b>1,000</b>   | early agents          |
| Q4 2026 | <b>10,000</b>  | agents + humans + IoT |
| Q1 2027 | <b>100,000</b> | every category        |

KNOWN RISKS

1. **Walled gardens may successfully wall.** Mitigation: target the long tail, not the giants.
2. **Byzantine forwarding faults at scale.** Eclipse, replay, and tampering are hardened today; silent "black-hole" relays remain the open frontier. Mitigation: heterogeneous-protocol simulator + forwarding-vitality route-around.
3. **Network effect may not compound before vendor stacks entrench.** Mitigation: faster cadence than vendor SDK iteration.

CONTACT

[axona.net](https://axona.net) · [davidasmith@gmail.com](mailto:davidasmith@gmail.com)